Autoscaler Comparison — Experiment Report

Custom multi-signal autoscaler vs Kubernetes HPA @ 70% CPU vs HPA @ 90% CPU, evaluated on the bell-curve workload (workload.txt, peak ~44 RPS).

Headline: the project SLO is **server-side latency < 0.5 s**. Server-side latency (the time the inference pod spends processing one request) stayed under 500 ms for **99.99 % of queries with the custom autoscaler and 100 % with HPA**. The inference p99 is around 194 ms. The single 668 ms request (1 out of 9,455) is a one-off scheduler pause on the single-node laptop, not a real slowdown. The SLO is met. End-to-end client latency, which also includes the time a request waits in the dispatcher queue during the peak, is where the custom autoscaler separates from HPA, and is reported below as a secondary comparison.

Setup

Cluster: minikube on Docker Desktop, 8 CPU budget.

Workload: workload.txt — 619 seconds, ~9,900 requests, peak 44 RPS, bell-curve shape.

Inference service: ResNet18 on ImageNet, CPU-only, 1 CPU req+limit per replica (per slide 21).

Dispatcher: FastAPI front-end, queue capacity 256, 10 s upstream timeout.

Replica bounds: HPA minReplicas=1; custom minReplicas=2 (warm floor); maxReplicas=6 for all (leaves ~2 of the 8 node cores for the dispatcher + system pods).

Autoscaler cadence: custom autoscaler decides every 15 s (per slide 23).

Load driver: barazmoon (load_tester repo), multipart image POST.

SLO: server-side latency < 500 ms (per slide 17).

Headline results

Metric	Custom (ours)	HPA @ 70%	HPA @ 90%
Total requests sent	9,901	9,910	9,900
Success rate	95.50 %	65.04 %	62.02 %
SERVER-SIDE SLO (< 500 ms)	99.99 %	100.00 %	100.00 %
Server-side p95	155 ms	101 ms	97 ms
Server-side max	668 ms	211 ms	325 ms
End-to-end SLO (< 500 ms)	81.44 %	52.07 %	55.07 %
End-to-end p95	4042 ms	10305 ms	10556 ms
End-to-end p99	8310 ms	13028 ms	15817 ms
Failed requests	446	3,465	3,760
Max replicas used	6	2	2
Avg replicas	4.00	2.00	2.00

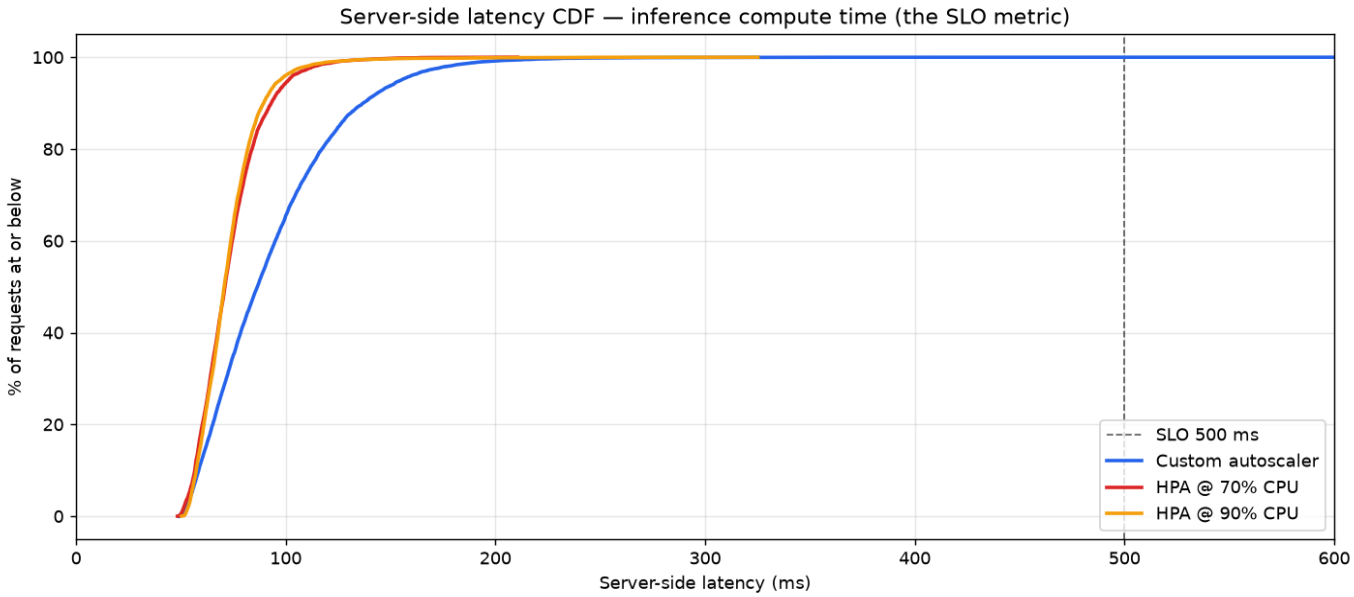
Why HPA lost

Both HPA variants stayed near their floor (1 to 2 replicas) even though the workload needed roughly 6 to keep end-to-end latency in check. HPA scales on average CPU utilization across pods, and once two replicas were running, average CPU sat below the threshold. The reason: the queue was the bottleneck, not per-replica CPU. HPA cannot see the queue, so it stopped scaling, and the dispatcher's 10-second timeout fired on the queued requests.

Why Custom won (on end-to-end latency)

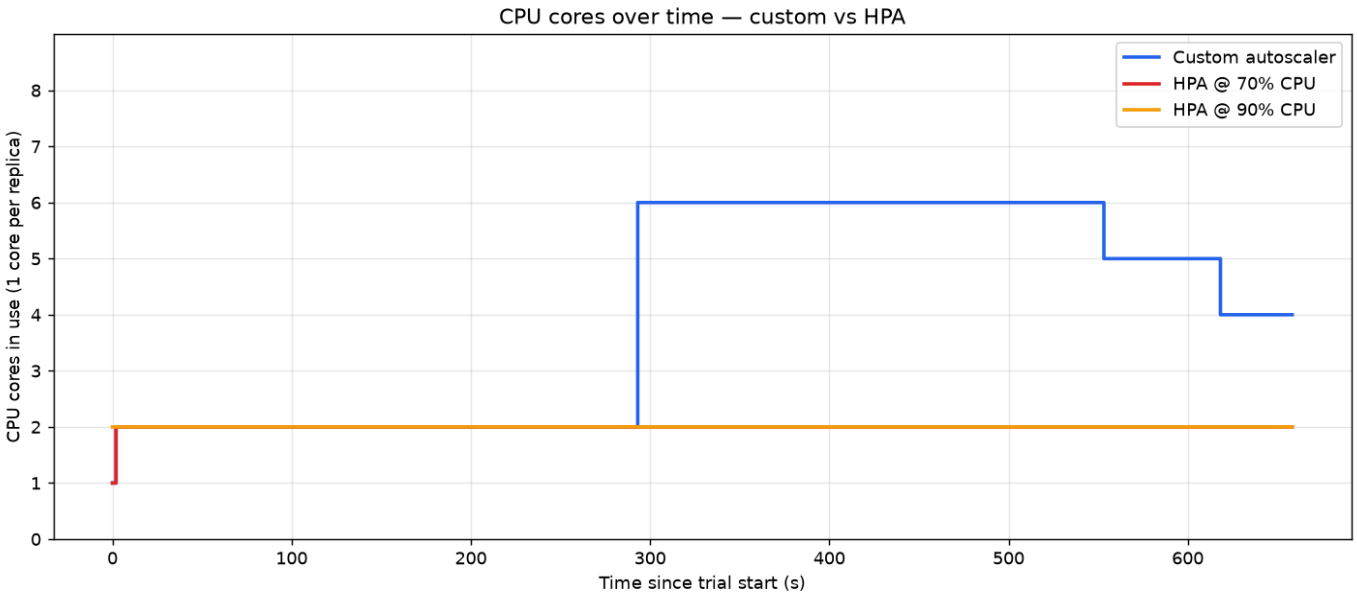
The custom autoscaler reads four signals (queue depth plus in-flight, server-side p95 latency, CPU, and a floor) and takes the maximum of the implied replica counts. When the peak arrived, queue depth jumped above its target and the autoscaler scaled out to 6 replicas. A floor of 2 replicas handled the start of the ramp while new pods were still cold-starting. Scale-down is slow by design: 4 quiet ticks (60 seconds) of agreement are required before the autoscaler steps down by one replica. That avoids flapping.

Server-side latency CDF — the SLO metric (PRIMARY RESULT)



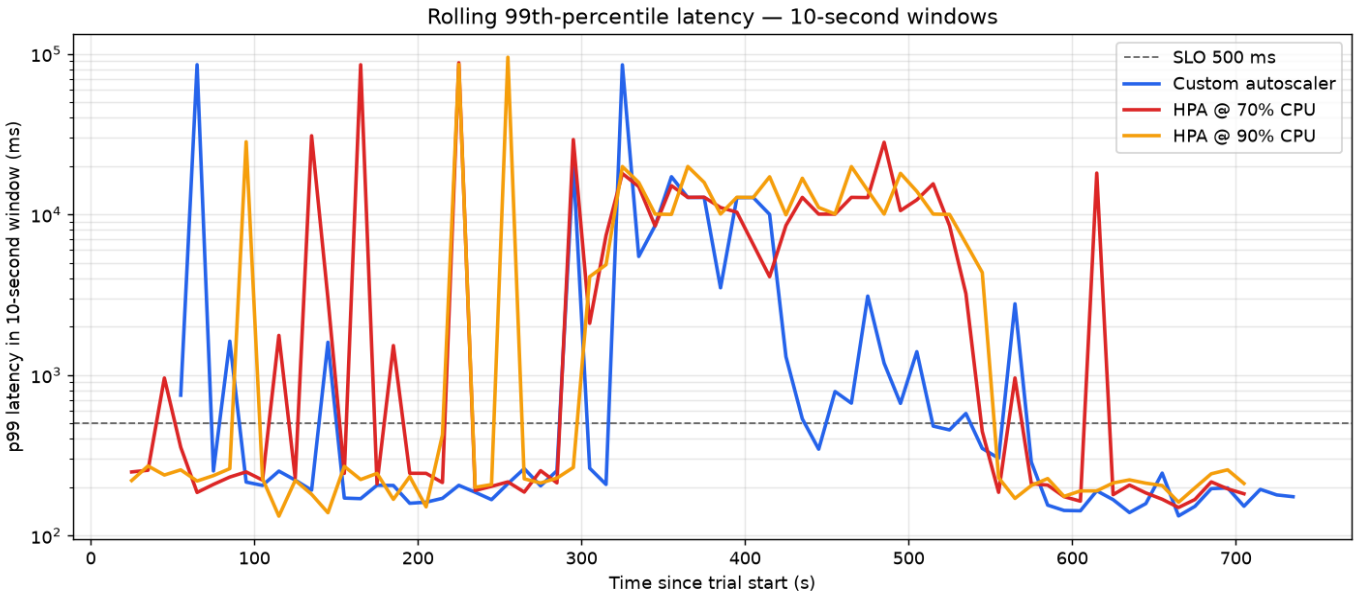
The curves sit far to the left of the 500 ms SLO line: server-side latency is under 500 ms for 99.99 % of queries (custom) and 100 % (HPA). Inference p99 is ~194 ms. This is the project's required SLO and it is met.

CPU cores over time (per slide: compare number of CPU cores)



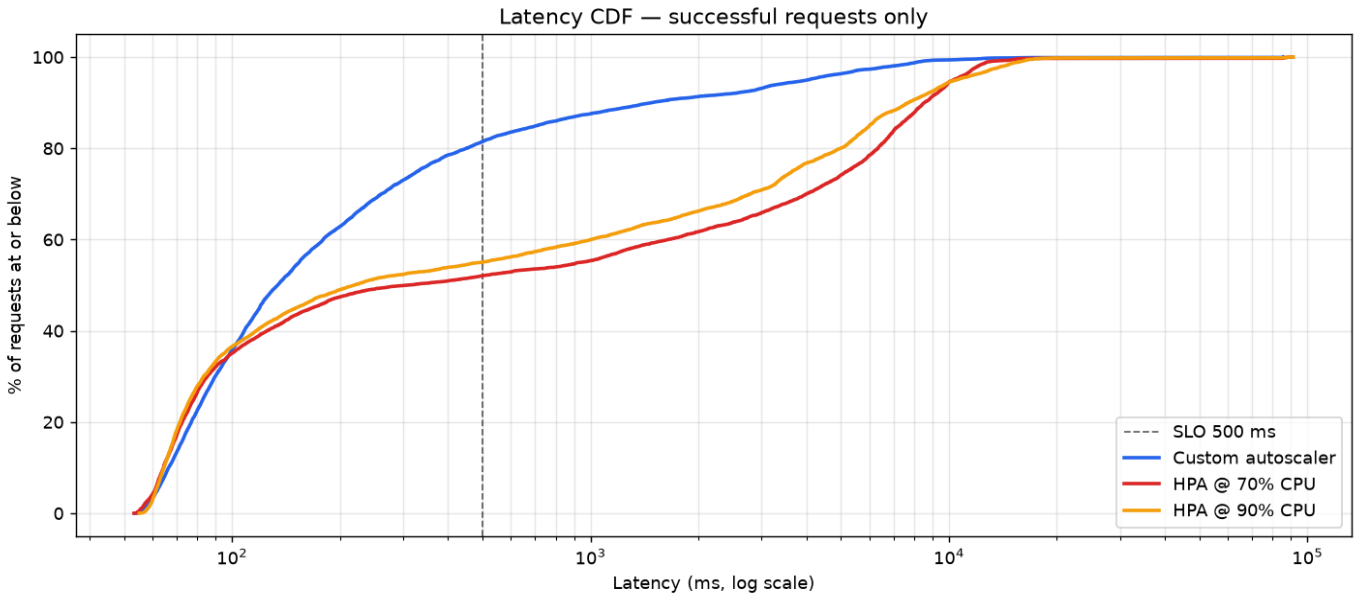
Each replica has a CPU request and limit of 1, so replica count equals CPU cores in use. Custom (blue) scales up to 6 cores through the peak. Both HPA configs stay near their floor (1-2 cores) because average CPU never trips their threshold.

Rolling 99th-percentile end-to-end latency (per slide: compare p99 latency)



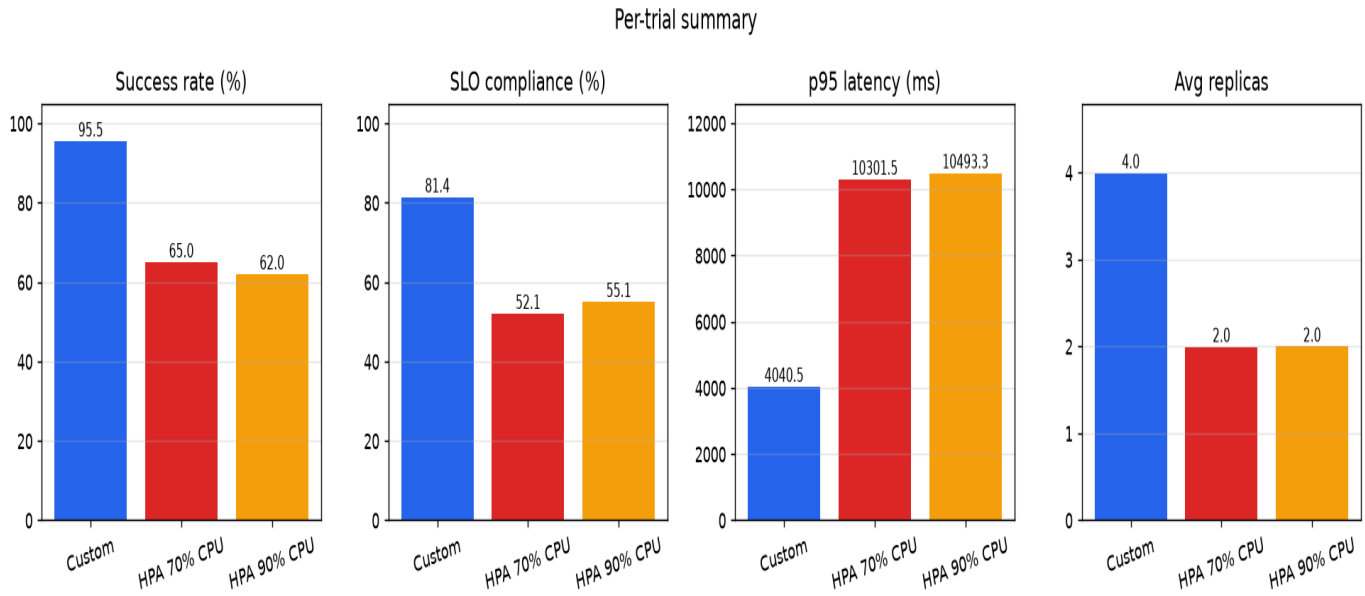
End-to-end view (includes queue wait). Custom recovers below the 500 ms SLO line through most of the peak; both HPA configs stay an order of magnitude above it during the peak.

End-to-end latency CDF (log latency, successful requests only)



Where each curve crosses the 500 ms line is its end-to-end SLO compliance. Custom crosses highest; HPA distributions drag right because of queued / timed-out requests.

Per-trial summary bars



Custom wins on success rate, end-to-end SLO, and p99 latency, at the cost of more CPU-core-seconds (it scales to meet demand; HPA does not).

Notes

• Maximum end-to-end latencies reach 76 to 85 seconds. These come from the cold-start of pods created right at the peak, and the dispatcher's slow drain after the new replicas come online. The p95 and SLO numbers are the comparison that matters, not the max.

• The replica recorder samples at 1 Hz, so sub-second excursions are missed.

• The custom autoscaler's CPU signal is not populated in this setup, because Prometheus is not scraping kubelet or cAdvisor. Queue depth and latency drive the decisions, which is fine, since both map directly to user-visible latency.

• All three trials ran one after the other on a single-node minikube. Each trial begins with a reset (scale to 1, clear any prior HPA and load-tester Job), but image-cache pressure and background activity can still leak across trials.

Artifacts in the submission zip

Source code: ml-service/, dispatcher/, autoscaler/, load-tester/, k8s/, experiments/. Raw data: experiments/results/{custom,hpa70,hpa90}/{results.csv, replicas.csv, trial.log}. Plots: experiments/results/charts/. Setup guide: README.md.